Databases 2 - exam - February 12, 2024 - Dur. 2h

S. Comai, P. Fraternali, D. Martinenghi

A. Triggers (12 points)

Consider the following relational schema:

`PROJECT(PID, name, duration)`
`EMP(EID, name, salary)`
`ASSIGNMENT(PID, EID)`

where `PID` and `EID` in `ASSIGNMENT` are under foreign key constraints to `PROJECT` and `EMP`, respectively.

Your task is to define a set of triggers that maintain a table `BUDGET(PID, cost)` with the same content as would be maintained by the following view:

```
CREATE VIEW budget AS SELECT P.PID, COALESCE (SUM(salary), 0) AS cost
FROM project P LEFT JOIN assignment A ON P.PID = A.PID
               LEFT JOIN emp E ON E.EID = A.EID
GROUP BY P.PID;
```

where the `COALESCE()` function returns the first non-null value in its list of comma-separated arguments.

Assume that the primary key values are not updated and that each employee is assigned to no more than one project.

1. Complete the table by indicating, for each event and table, whether the maintenance of the content of the `BUDGET` table requires the implementation of a trigger. If so, write the name of the trigger (e.g., `T1`, `T2`, etc.), otherwise provide a justification (3 points).
2. Define the code of such triggers (9 points).

	PROJECT	EMP	ASSIGNMENT
INSERT	□ Yes – No □ Name:	□ Yes – No □ Name:	□ Yes – No □ Name:
UPDATE	□ Yes – No □ Name:	□ Yes – No □ Name:	□ Yes – No □ Name:
DELETE	□ Yes – No □ Name:	□ Yes – No □ Name:	□ Yes – No □ Name:

Solution.

	PROJECT	EMP	ASSIGNMENT
INSERT	Yes – Name: T1	No	Yes – Name: T4
UPDATE	No	Yes – Name: T3	No
DELETE	Yes – Name: T2	No	Yes – Name: T5

- There is no need of triggers for the updates on `PROJECT` and `ASSIGNMENT` tables because primary keys are not updated (see assumptions in the text).
- No trigger is required for insertions and deletions into the `EMP` table, as only assignments affect project costs.

1.
```
CREATE TRIGGER T1
AFTER INSERT ON PROJECT
FOR EACH ROW

INSERT INTO BUDGET
VALUES(NEW.PID, 0);
```

2.
```
CREATE TRIGGER T2
AFTER DELETE ON PROJECT
FOR EACH ROW

DELETE FROM BUDGET
WHERE PID = OLD.PID;
```

3.
```
CREATE TRIGGER T3
AFTER UPDATE OF SALARY ON EMP
FOR EACH ROW

UPDATE BUDGET
SET COST = COST + NEW.SALARY - OLD.SALARY
WHERE PID = (SELECT PID FROM ASSIGNMENT WHERE EID = NEW.EID);
```

4.
```
CREATE TRIGGER T4
AFTER INSERT ON ASSIGNMENT
FOR EACH ROW

UPDATE BUDGET
SET COST = COST + (SELECT SALARY FROM EMP WHERE EID = NEW.EID)
WHERE PID = NEW.PID;
```

5.
```
CREATE TRIGGER T5
AFTER DELETE ON ASSIGNMENT
FOR EACH ROW

UPDATE BUDGET
SET COST = COST - (SELECT SALARY FROM EMP WHERE EID = OLD.EID)
WHERE PID = OLD.PID;
```

B. Concurrency control (8 points)
Apply the distributed deadlock detection algorithm to the following wait-for conditions using two conventions:

1) a dependency $t_i \to t_j$ can be transmitted (forward) if $i > j$;
2) a dependency $t_i \to t_j$ can be transmitted (forward) if $i < j$;

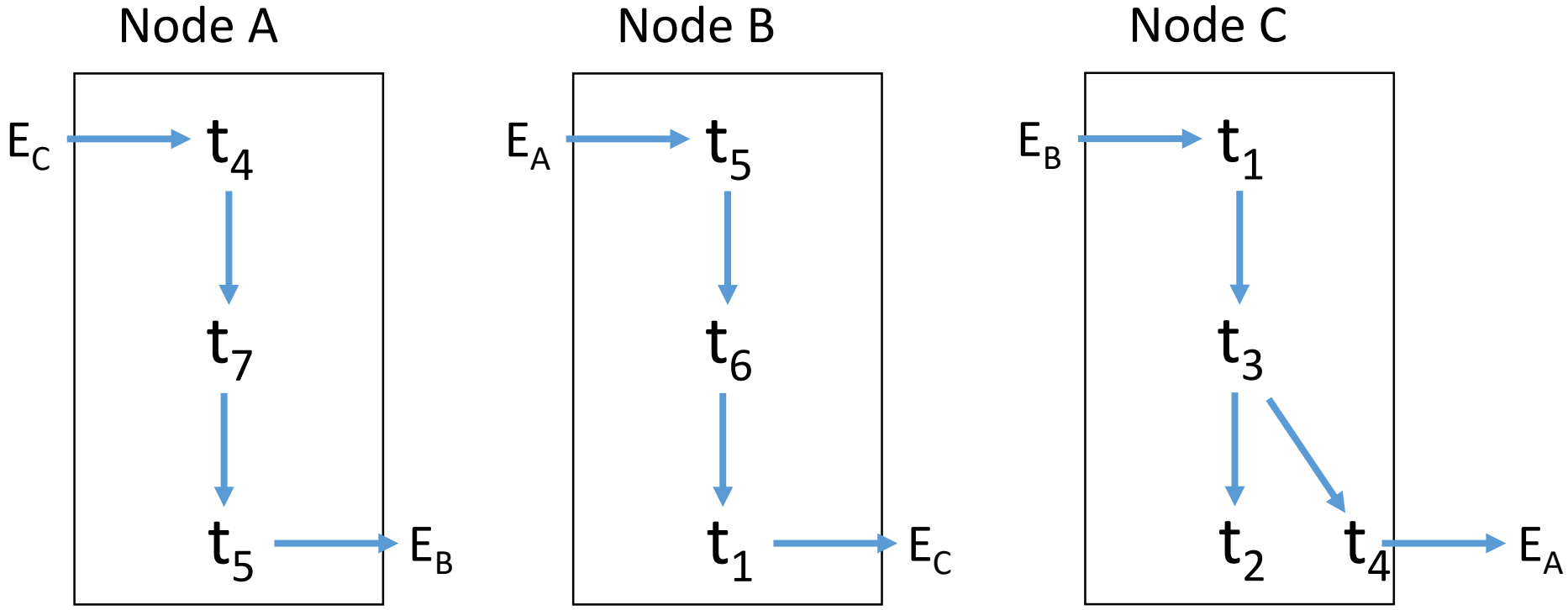
- Node A: $E_C \to t_4$; $t_4 \to t_7$; $t_7 \to t_5$; $t_5 \to E_B$;
- Node B: $E_A \to t_5$; $t_5 \to t_6$; $t_6 \to t_1$; $t_1 \to E_C$;
- Node C: $E_B \to t_1$; $t_1 \to t_3$; $t_3 \to t_2$; $t_3 \to t_4$; $t_4 \to E_A$.

For each convention, execute all the steps, reporting all the messages that are sent during the process.

Solution. See separate pdf file.

1) Solution

i>j



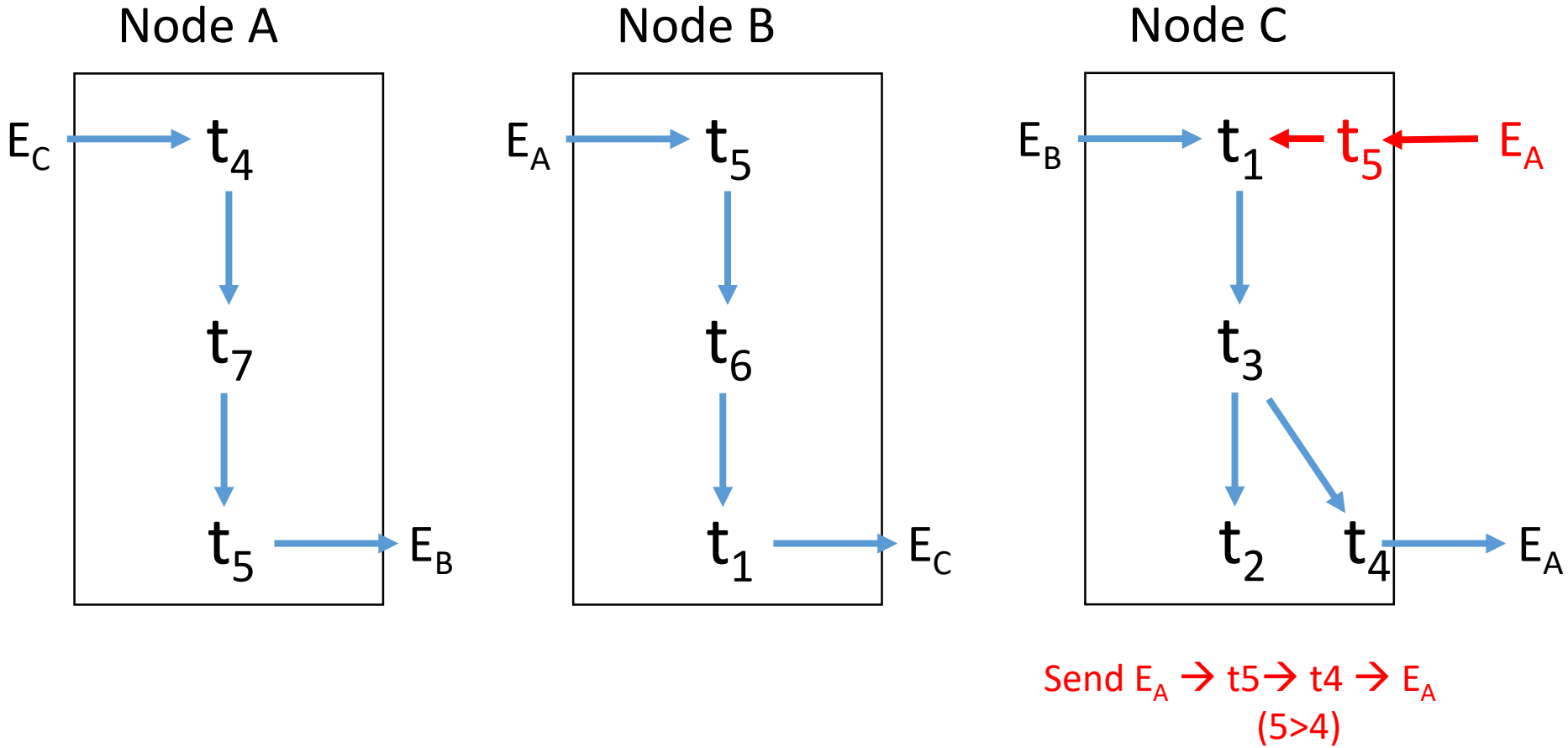
Node A cannot transmit (4<5)

Send E_A → t5 → t1 → E_C (5>1)

Node C cannot transmit (1<4)

Solution

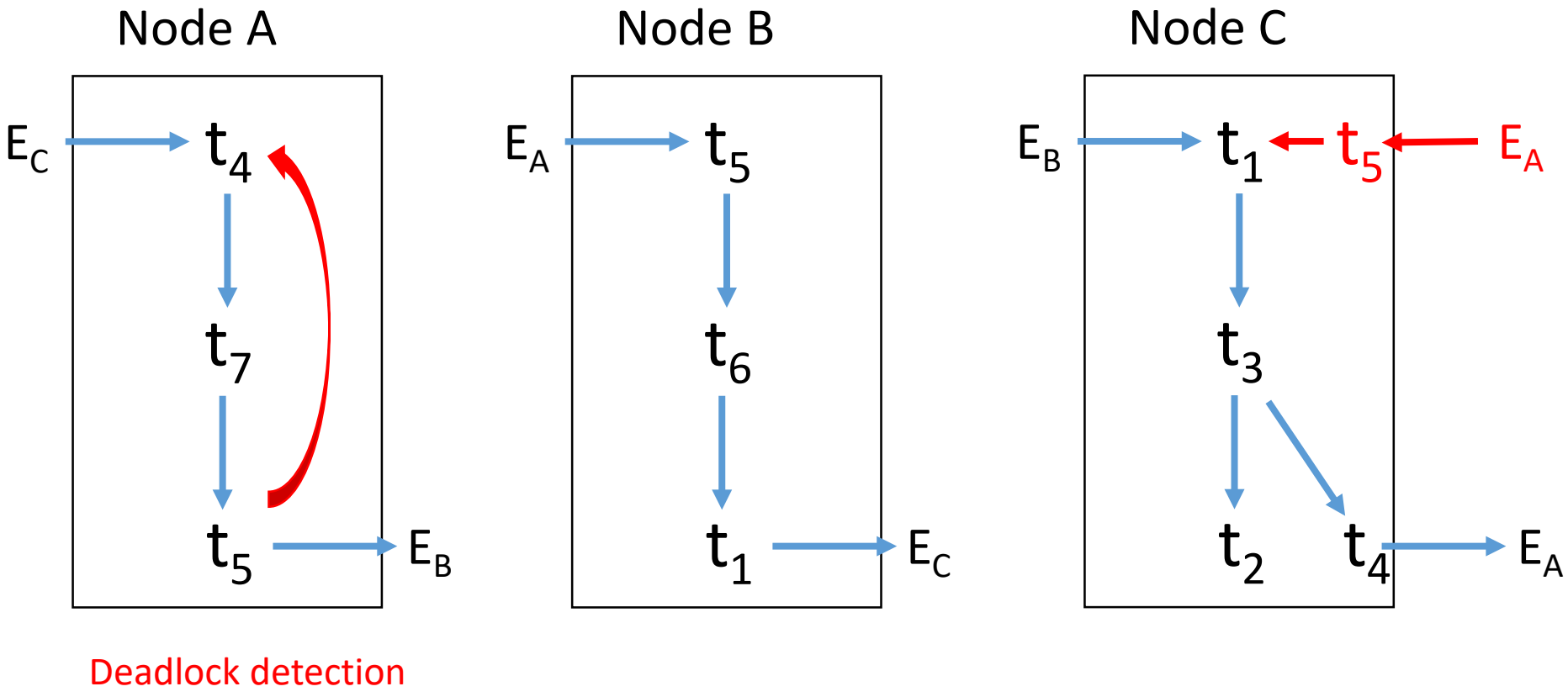
i>j



Send E_A → t5 → t4 → E_A
(5>4)

Solution

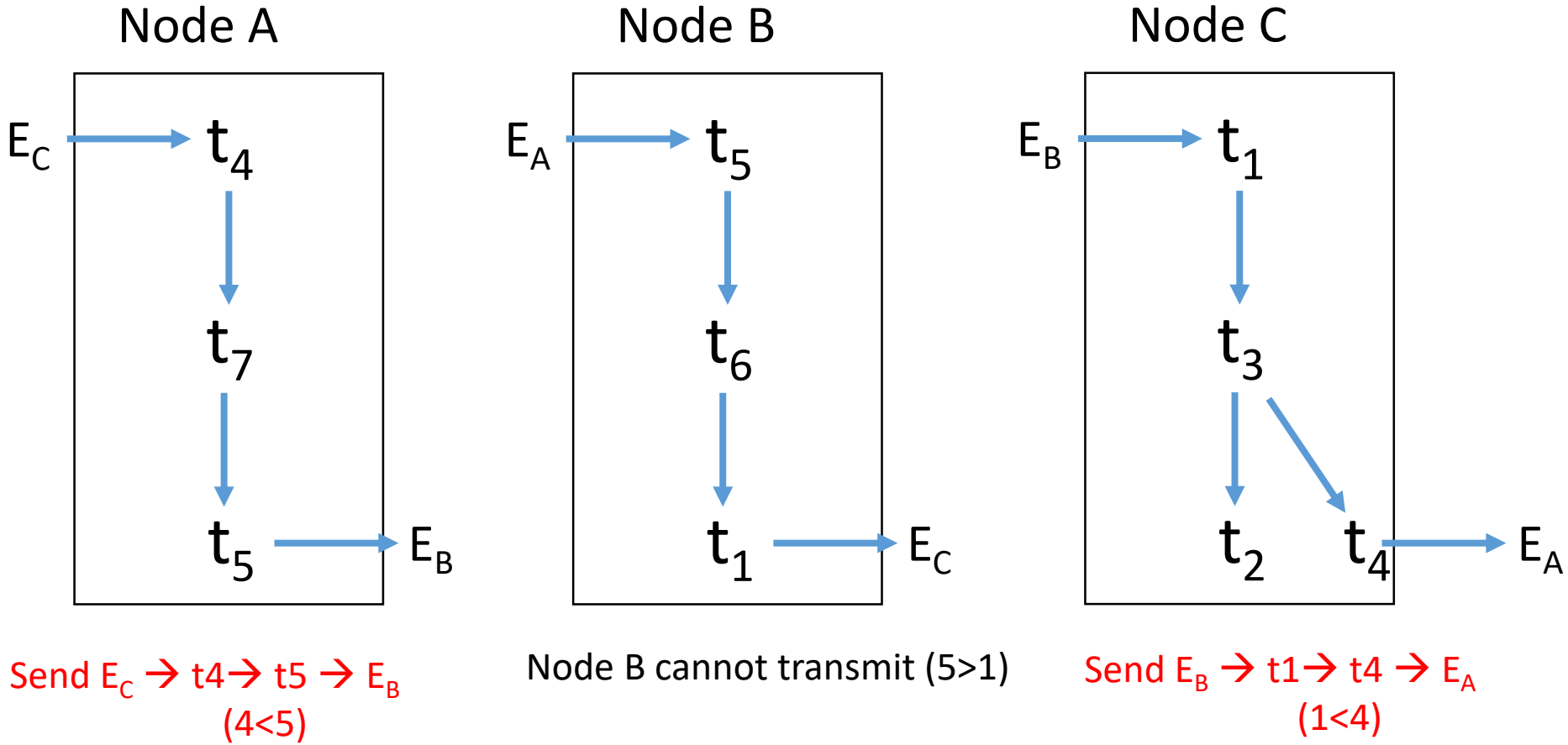
i>j



Deadlock detection

2) Solution

i<j



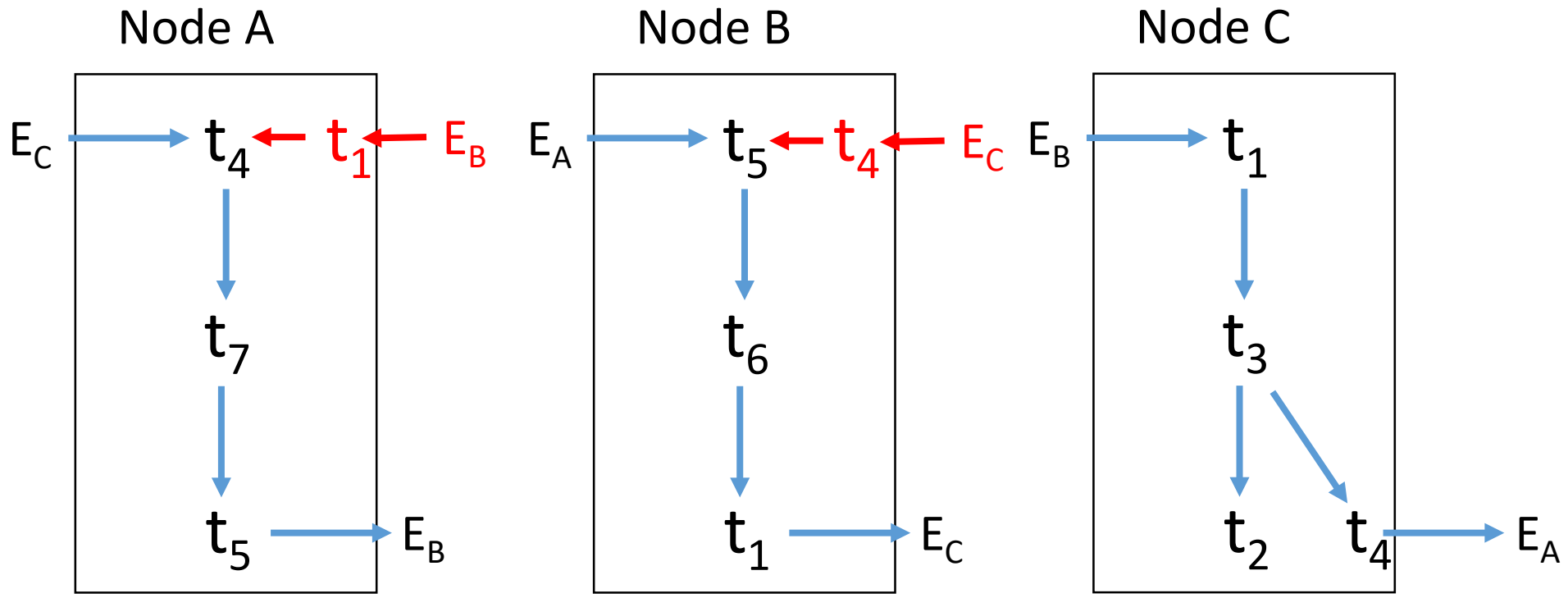
Send E_C → t4→ t5 → E_B
(4<5)

Node B cannot transmit (5>1)

Send E_B → t1→ t4 → E_A
(1<4)

Solution

i<j

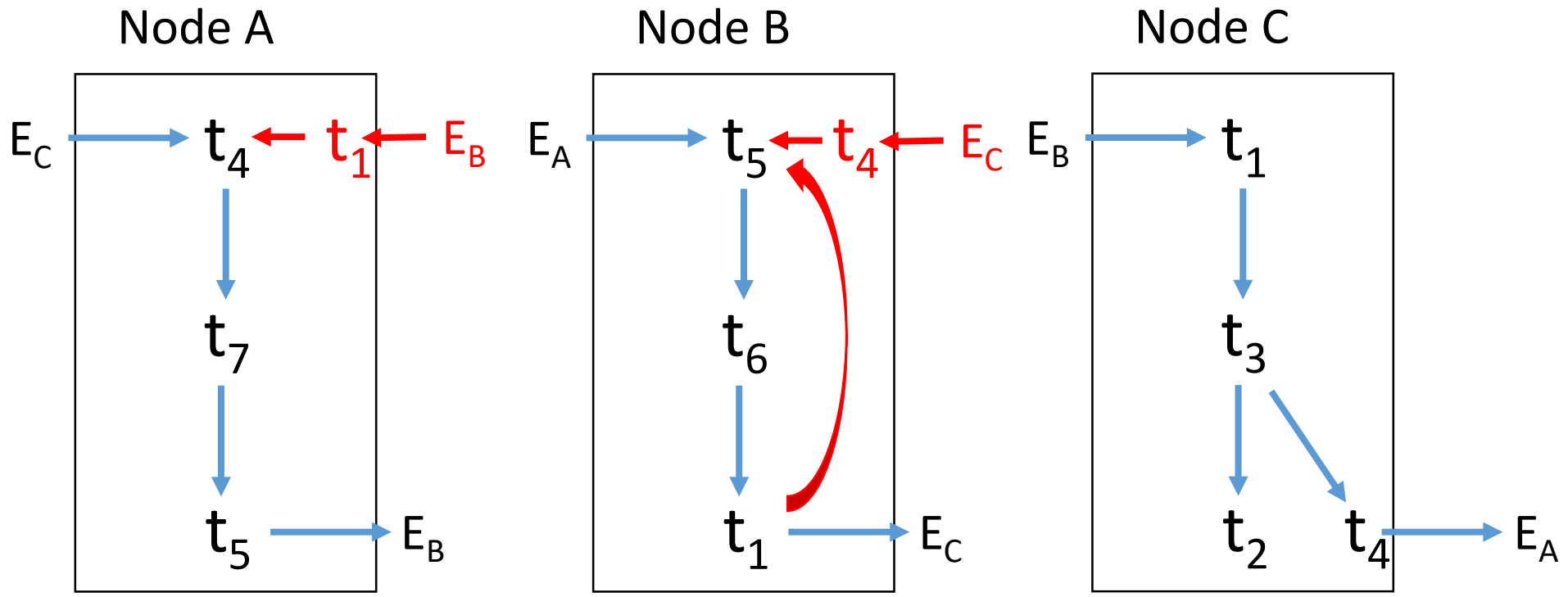


Send E_B → t1 → t5 → E_B
(1<5)

Node B cannot transmit (4>1)

Solution

i<j



Deadlock detection

C. Physical Databases (6+6=12 points)

Table `Task(tId, title, completion, wp)` contains 100K tuples stored in a primary hash structure built on `tId`, with 20K blocks and filling factor below 50%. Table `Workpackage(wpId, title, allocatedHours)` contains 5K tuples in 50 blocks stored in a primary entry-sequenced sequential structure. 70% of the tasks have more than 90% completion and 25% of the work packages have less than 100 allocated hours. Tasks are indexed by a B+ tree on the `wp` attribute (3 levels, 1000 leaf blocks). Suppose that you can cache up to 15 blocks.

Describe an efficient query plan for the following queries. Estimate their execution costs (**write the complete formula for the query**) and explain all the steps of the plan and their costs. The evaluation of the exercise takes into account also the degree of efficiency of the plan.

1.
```
SELECT * FROM Task     // nearly completed tasks of low effort work packages
WHERE completion > 90 and wp IN
  (SELECT wpId FROM Workpackage WHERE allocatedHours < 100)
```

2.
```
SELECT w.title, AVG(t.completion) // title and average completion across all tasks
FROM Workpackage w LEFT JOIN Task t ON t.wp=w.wpId
GROUP BY w.wpId, w.title;
```

Solution.

See separate pdf file.

Statistics

- Table `Task(tId, title, completion, wp)`
  - 100K tuples
  - **primary** hash structure on tId with **20K blocks** and filling factor < 50%, **5 tuples per block**, no overflow cost
  - Secondary B+ index on wp attribute (3 levels, 1000 leaf blocks, 100 pointers to task per leaf).
  - 70K tuples with more than 90% completion
- Table `Workpackage(wpId,title,allocatedHours)`
  - 5K tuples
  - **primary entry-sequenced** sequential structure of 50 blocks
  - 100 tuples per block
  - 1250 tuples with less than 100 allocated hours (13 blocks: cacheable)
- 20 tasks per workpackage
- Max cache size = 15 blocks

Query 1

```
SELECT * FROM Task
WHERE completion > 90 and
wp IN (SELECT wpId
FROM Workpackage WHERE allocatedHours < 100)
```

Equivalent to

```
SELECT * FROM Task T, Workpackage W
WHERE T.completion > 90 and
T.wp= W.wpId AND W.allocatedHours < 100
```

Query 1

```
SELECT * FROM Task
WHERE completion > 90 and
wp IN (SELECT wpId
FROM Workpackage WHERE
allocatedHours < 100)
```

- Strategy 1
  - Scan workpackage and filter+cache 1250 tuples: cost = 50 I/O accesses
  - Lookup B+ tree on wp for the retrieved tuples: cost = 1250 * (3 levels + 20 task data access per workpackage) = 28,75K I/O accesses
  - Total cost = 50 + 1250*23 = 50 + 28750= **28,8k** I/O accesses
- Strategy 2
  - Scan workpackage and filter+cache 1250 tuples: cost = 50 I/O accesses
  - Sort 1250 tuples by wpId in main memory: 0
  - Merge scan on wpId: cost = 0 (main memory) + (2 intermediate nodes + 1000 leaf blocks)
  - Access relevant task tuple per each relevant workpackage: cost = 1250*20 I/O accesses
  - Total cost = 50 + 1002 + 1250*20 = **26k** I/O accesses
- Stategy 3
  - Scan workpackage and filter+cache 1250 tuples : cost = 50 I/O accesses
  - Scan tasks (filter tasks with completion > 90) and compute the join with the cached workpackages (nested loop):

    Total cost = 50 + 20k = **20K** I/O accesses

Query 2

```
SELECT w.title, average(t.completion)
FROM Workpackage w
   LEFT JOIN Task t ON
   t.wp=w.wpId
GROUP BY w.wpId;
```

The left join returns all the tuples of the Workpackage table and the matching tuples from the Task table.

- Scan workpackage: cost = 50 I/O accesses
- Lookup task through the B+ tree on wpId for all the workpackages (B+ tree + follow the pointers to retrieve the full task tuples): cost = 5k * (3 levels + 20 data access for relevant tasks per workpackage)
- Total cost = 50 + 5K*(3+20) = **115k** I/O accesses